

HANGA WORKS

2-Week Sprint Plan — v2.0 (Updated)

Phase 1 · 4 Developers · Node.js · React · PostgreSQL · GitHub · Render · Vercel

1. Technology Stack

Layer	Technology	Details & Rationale
Frontend	React 18 + TypeScript	Vite build tool, React Router v6, TanStack Query, Tailwind CSS. Deployed to Vercel with automatic preview per PR.
Backend API	Node.js + NestJS + TypeScript	NestJS modular architecture with decorators, DI, and Guards for RBAC. OpenAPI/Swagger docs. REST at /api/v1/.
Database	Prisma + PostgreSQL	Prisma schema as single source of truth. Migrations via prisma migrate. Hosted PostgreSQL on Render.
Auth	JWT + Passport.js	passport-jwt strategy. Access tokens 15 min, refresh tokens 7 days in httpOnly cookies. RBAC Guards.
Cache	Redis (ioredis)	Session management, rate limiting, course listing cache, notification queues.
Real-time	Socket.io	WebSocket server on NestJS. socket.io-client on React for live notifications and status updates.
Storage	S3 / Cloudflare R2	Certificate PDFs, course media, profile photos. Presigned URLs for secure upload.
Email	SendGrid	Transactional emails: registration, application updates, course completion, cert issued.
CI/CD	GitHub Actions	On PR: tsc, eslint, jest. On merge to main: auto-deploy to Render (API) and Vercel (Web).

2. Repository & Project Structure

Two separate GitHub repositories under the team organisation:

hanga-works-backend (NestJS / Render)	hanga-works-frontend (React / Vercel)
src/auth/ ← JWT, Passport, Guards src/users/ ← User module + RBAC src/lms/ ← Courses, Progress, Quiz src/jobs/ ← Listings, Applications src/certifications/ ← Issue, Verify, PDF src/notifications/ ← SendGrid, Socket.io src/employer/ ← ATS, Dashboard src/analytics/ ← Metrics endpoints src/prisma/ ← Prisma service src/redis/ ← Redis service src/common/ ← Filters, Pipes prisma/schema.prisma prisma/seed.ts .env.example	src/components/ui/ ← Shared UI components src/components/layout/ ← Sidebar, Topbar src/components/shared/ ← ProgressBar, Avatar src/pages/auth/ ← Login, Register src/pages/dashboard/ ← Dashboard src/pages/courses/ ← CourseList, Detail src/pages/jobs/ ← JobList, Apply src/pages/profile/ ← Profile page src/pages/employer/ ← Employer pages src/features/ ← Feature hooks src/services/ ← API service layer src/types/ ← TypeScript types src/utils/ ← Helper functions .env.example

3. Updated Team Assignments

Roles have been updated from the original sprint plan v1.1:

YVES <i>Backend Developer</i> • Prisma schema & seed data • LMS module (courses, progress, quiz) • Jobs marketplace API • Certifications module • Analytics & job matching • Redis caching (ioredis) • Swagger / OpenAPI docs	LEON <i>Backend Developer</i> • NestJS scaffold & setup • JWT / Passport auth + RBAC • Notifications (SendGrid + Socket.io) • Employer ATS module • Security hardening • Production deploy (Render) • Database migrations	ROSINE <i>Frontend Developer</i> • UI template & design reference • Auth screens (login, register) • LMS UI (courses, progress) • Job marketplace UI • Skill profile UI • UI polish & mobile responsive • Accessibility (WCAG 2.1 AA)	BLAISE <i>Frontend Developer</i> • React scaffold + Tailwind setup • Dashboard layout + sidebar nav • GitHub Actions CI/CD • Employer dashboard + ATS kanban • Admin panel • Lighthouse performance audit • Vercel production deploy
---	---	---	--

4. Week 1 — Foundation & Design (Days 1–5)

Day 1 — Kickoff & Scaffold

Day 1	ALL	[MEETING] Project kickoff (2 hours — all hands) - Align on SRS Phase 1 scope and folder structure - Agree on TypeScript strict mode, ESLint/Prettier config - Confirm Git branching strategy: feature/task-name → develop → main - Set daily standup cadence (15 min) - Each person confirms their Day 1 task is clear
Day 1	LEON	[DEV] NestJS scaffold + full project structure - Run: <code>npx @nestjs/cli new hanga-works-backend --strict</code> - Create ALL folders and empty placeholder files per agreed structure - Set up .env file with all required variables - Connect Prisma to PostgreSQL - Push to GitHub main branch + create develop branch - Add branch protection rules on main
Day 1	YVES	[DEV] Prisma schema + seed data - Define all models: User, Course, Module, Enrollment, Progress - Define: Job, Application, Certificate, SkillTag models - Set up all relations between models - Write seed.ts: 20 learners, 5 employers, 10 courses, 20 jobs - Run: <code>npx prisma migrate dev --name init</code> - Verify with: <code>npx prisma studio</code>

Day 1	BLAISE	[DEV] React scaffold + Tailwind + routing Run: npm create vite@latest hanga-works-frontend -- --template react-ts Install: tailwindcss, react-router-dom, @tanstack/react-query, axios Install: socket.io-client Set up all routes in App.tsx Create src/services/api.ts (axios instance) Push to GitHub + link repo to Vercel
Day 1	ROSINE	[DESIGN] Find UI template + design reference Go to figma.com/community Search: job board dashboard or LMS dashboard Duplicate a free template to team Figma Share Figma link + template URL with team on WhatsApp No need to design from scratch — use existing template

Days 2–3 — Auth, LMS API & CI/CD

Day 2-3	LEON	[DEV] Auth module — JWT + Passport + RBAC Files: src/auth/auth.module.ts, auth.controller.ts, auth.service.ts Files: src/auth/strategies/jwt.strategy.ts Files: src/auth/guards/jwt-auth.guard.ts, roles.guard.ts Files: src/auth/decorators/roles.decorator.ts Files: src/auth/dto/register.dto.ts, login.dto.ts Endpoints: POST /auth/register, /login, /refresh, /logout Access token: 15 min Refresh token: 7 days httpOnly cookie
Day 2-3	YVES	[DEV] LMS NestJS module — courses, enrollment, progress Files: src/lms/courses/, src/lms/enrollment/, src/lms/progress/ Endpoints: GET /courses, GET /courses/:id Endpoints: POST /enrollments, PATCH /progress/:moduleId Endpoints: POST /quiz/:id/submit Redis cache for course listing (TTL 5 min) Write Jest unit tests Branch: feature/lms-module
Day 2-3	ROSINE	[DEV] Auth screens + dashboard shell (React) Files: src/pages/auth/Login.tsx, Register.tsx, VerifyEmail.tsx Files: src/features/auth/useAuth.ts Files: src/services/auth.service.ts Protected routes with auth context (React Router v6) Connect to Leon's auth API endpoints Branch: feature/auth-pages
Day 2-3	BLAISE	[DEV] Dashboard layout + sidebar nav Files: src/components/layout/Sidebar.tsx, Topbar.tsx, DashboardLayout.tsx Files: src/components/ui/Button.tsx, Input.tsx, Card.tsx, Badge.tsx Files: src/components/shared/NotificationBell.tsx, Avatar.tsx Files: src/pages/dashboard/Dashboard.tsx

		TanStack Query setup + axios instance Branch: feature/dashboard-layout
--	--	---

Day 4 — Mid-Sprint Sync & Features

Day 4	ALL	[MEETING] Mid-sprint sync (1 hour) Review Swagger UI at /api/docs Finalise request/response shapes for Job and LMS endpoints Rosine demos React screens on Vercel preview URL Identify and resolve any blockers
-------	-----	--

Day 4	YVES	[DEV] Jobs marketplace API Files: src/jobs/jobs.module.ts, jobs.controller.ts, jobs.service.ts Files: src/jobs/dto/create-job.dto.ts, apply-job.dto.ts Endpoints: POST /jobs, GET /jobs (filters), GET /jobs/:id Endpoints: POST /jobs/:id/apply, PATCH /applications/:id/status DTOs with class-validator Branch: feature/jobs-api
-------	------	--

Day 4	LEON	[DEV] Notifications — SendGrid + Socket.io Files: src/notifications/notifications.module.ts Files: src/notifications/notifications.gateway.ts Files: src/notifications/notifications.service.ts SendGrid: registration, application update, course complete, cert issued Socket.io events: job-match, application-status, course-complete Branch: feature/notifications
-------	------	--

Day 4	ROSINE	[DEV] LMS UI — course browser + progress Files: src/pages/courses/CourseList.tsx, CourseDetail.tsx Files: src/components/shared/ProgressBar.tsx Files: src/features/lms/useCourses.ts, useProgress.ts Connect to Yves's LMS API endpoints Branch: feature/lms-ui
-------	--------	--

Day 4	BLAISE	[DEV] GitHub Actions CI/CD + Vercel File: .github/workflows/ci.yml On PR: run tsc --noEmit and eslint On merge to main: Vercel auto-deploys Set VITE_API_URL in Vercel dashboard Branch protection rules on main confirmed Branch: feature/cicd
-------	--------	--

Day 5 — Week 1 Wrap & Staging Deploy

Day 5	ALL	[REVIEW] PR reviews + integration merge + staging deploy
-------	-----	---

		All feature branches open PRs to develop CI must pass (tsc, eslint, jest) + 1 approving review Merge to develop → auto-deploy to Render staging + Vercel preview Smoke test: register → login → enrol course → job browse
--	--	---

Day 5	ALL	[MEETING] Week 1 retro + Week 2 planning (1 hour) Demo on Vercel preview and Render staging URLs Velocity check vs plan — reprioritise if needed Confirm Week 2 task assignments Update Trello board
-------	-----	--

✓ Week 1 Milestone — Auth API live on Render staging · LMS + Jobs API on Swagger · React app on Vercel preview · GitHub Actions CI green

5. Week 2 — Features, Polish & Deploy (Days 6–10)

Day 6 — Certifications & Marketplace UI

Day 6	YVES	[DEV] Certifications module — PDF + S3 Files: src/certifications/certifications.module.ts Files: src/certifications/certifications.controller.ts Files: src/certifications/certifications.service.ts Auto-trigger on course completion (NestJS EventEmitter) Generate PDF with pdfkit, upload to Cloudflare R2 / AWS S3 GET /certificates/verify/:token (public) POST /certificates/validate (employer-facing) Branch: feature/certifications
Day 6	LEON	[DEV] Employer ATS module — pipeline stages Files: src/employer/employer.module.ts Files: src/employer/employer.controller.ts, employer.service.ts Files: src/employer/dto/update-stage.dto.ts Endpoints: POST /employer/jobs, GET /employer/jobs/:id/applicants Endpoints: PATCH /employer/applications/:id/stage Stages: applied → reviewing → shortlisted → hired/rejected Protected by @Roles('EMPLOYER') guard Branch: feature/employer-ats
Day 6	ROSINE	[DEV] Job marketplace UI Files: src/pages/jobs/JobList.tsx, JobDetail.tsx, MyApplications.tsx Files: src/services/jobs.service.ts, src/types/job.types.ts Files: src/features/jobs/useJobs.ts, useApplications.ts Job listing with filters, one-click apply Application status: Applied, Reviewing, Shortlisted, Hired, Rejected Branch: feature/jobs-ui
Day 6	BLAISE	[DEV] Employer dashboard + ATS kanban Files: src/pages/employer/EmployerDashboard.tsx Files: src/pages/employer/PostJob.tsx, Applicants.tsx Files: src/components/ui/Modal.tsx Kanban board: Applied → Reviewing → Shortlisted → Hired Candidate profile modal/drawer Protected: EMPLOYER role only Branch: feature/employer-ui

Days 7–8 — Analytics, Profiles & Admin

Day 7-8	YVES	[DEV] Analytics endpoints + job matching score Files: src/analytics/analytics.module.ts Files: src/analytics/analytics.controller.ts, analytics.service.ts
---------	------	---

		GET /analytics/overview — DAU, MAU, completion rate GET /jobs/recommended — match user skills to jobs, return top 10 GET /analytics/export?format=csv Branch: feature/analytics
--	--	--

Day 7-8	ROSINE	[DEV] Skill profile UI + UI polish Files: src/pages/profile/Profile.tsx Files: src/utils/formatDate.ts, formatCurrency.ts Skill tags with proficiency selector Certificate badges display Empty state screens for no jobs / no courses Loading skeleton screens Mobile responsive check on all pages Branch: feature/profile-ui
---------	--------	--

Day 7-8	BLAISE	[DEV] Admin panel + hooks Files: src/pages/admin/AdminPanel.tsx Files: src/hooks/useLocalStorage.ts, useSocket.ts User list with account actions Platform stats cards Content moderation queue Branch: feature/admin-panel
---------	--------	---

Day 9 — QA, Security & Performance

Day 9	ALL	[REVIEW] Full QA pass — all flows end-to-end on staging Test: register → login → enrol course → complete → cert issued Test: apply to job → employer sees applicant in ATS Test on mobile viewport (Chrome DevTools) Log bugs in GitHub Issues with severity labels
-------	-----	--

Day 9	LEON	[DEV] Security hardening Files: src/common/filters/http-exception.filter.ts Files: src/common/interceptors/response.interceptor.ts Files: src/common/pipes/validation.pipe.ts Add helmet middleware (HTTP security headers) Add @nestjs/throttler rate limiting: 100 req/15 min Verify no secrets in GitHub codebase (git grep) Branch: feature/security
-------	------	--

Day 9	BLAISE	[DEV] Performance audit Lighthouse audit on Vercel preview (target: Performance > 90) Code-split heavy routes with React.lazy() Verify WebSocket reconnection logic Accessibility check with Axe
-------	--------	---

Day 10 — Production Deploy & Demo

Day 10	LEON	[DEVOPS] Production deployment — Render Merge develop → main (CI green + 2 approvals) Set all env vars in Render dashboard Run: npx prisma migrate deploy on production DB Verify HTTPS on Render URL Confirm daily DB backup active
--------	------	--

Day 10	BLAISE	[DEVOPS] Production deployment — Vercel Vercel auto-deploys from main branch Set all production env vars in Vercel dashboard Confirm frontend connects to Render API Final mobile sweep
--------	--------	--

Day 10	ALL	[MEETING] Final demo & handoff presentation (2 hours) Yves: API demo via Swagger UI Leon: Auth flow + security + Render deployment overview Rosine: Full UI walkthrough on Vercel production URL Blaise: CI/CD pipeline + employer dashboard demo Write README.md for both repos Document Phase 2 backlog in GitHub Issues
--------	-----	---

✓ Week 2 Milestone — HANGA WORKS Phase 1 LIVE: API on Render · Frontend on Vercel · PostgreSQL migrated · CI/CD green · All features shipped

6. CI/CD Pipeline Summary



Environment variables are never committed to GitHub. Set in Render dashboard (API) and Vercel dashboard (Web) only.

7. Non-Functional Requirements

Category	Requirement	Target	How We Meet It
Performance	Page load (P95)	< 2 seconds	Vercel CDN + Vite code-splitting + React.lazy()
Performance	API response	< 500ms	Redis cache + Prisma connection pool
Performance	WebSocket latency	< 1 second	Socket.io on Render (same region as DB)

Security	Data in transit	TLS 1.2+	Render + Vercel enforce HTTPS automatically
Security	Auth	JWT + refresh	Passport.js strategy, httpOnly cookies
Security	Input validation	All endpoints	class-validator on all DTOs
Scalability	Concurrent users	10,000+	Stateless NestJS + Render autoscale
Reliability	Uptime SLA	99.5%	Render managed infra + daily PG backups
Usability	Onboarding	< 5 min	3-step wizard, < 5 form fields per step
Usability	Accessibility	WCAG 2.1 AA	Tailwind a11y utilities + Axe CI scan

8. Sprint KPIs

30+	15+	2	90+
API endpoints (Swagger)	React pages / routes	GitHub Actions workflows	Target Lighthouse score